

RecoMart Recommendation Pipeline

Data Management for Machine Learning - Assignment I
End-to-End Data Management Pipeline for a Recommendation System

This PDF is the single consolidated documentation report. It covers problem formulation, data ingestion, raw storage, validation, preparation, EDA, SQL transformation, feature store, DVC versioning, MLflow tracking, model training, inference, orchestration, and reproducibility.

1. Assignment Requirement Coverage

Requirement	Status	Evidence
Problem formulation	Covered	Business problem, objectives, outputs, metrics
Data collection and ingestion	Covered	DummyJSON API and Retailrocket CSV data
Raw data storage	Covered	Local data lake layout with source/type/date structure
Data profiling and validation	Covered	Automated validation scripts and reports
Data preparation and EDA	Covered	Prepared Parquet datasets and summary plots
Feature engineering and transformation	Covered	DuckDB SQL warehouse and feature tables
Feature store	Covered	Custom registry and retrieval demo
Data versioning and lineage	Covered	DVC metadata and versioning workflow
Model training and evaluation	Covered	Retailrocket popularity baseline and content-based recommender with HitRate, ...
Pipeline orchestration	Covered	Prefect flows and successful run logs
Documentation and demo	Covered	Single PDF report, screenshots, video script

2. Problem Formulation

RecoMart is an e-commerce startup that wants product recommendations to improve customer engagement, conversion, and cross-selling. The pipeline produces clean datasets, engineered features, trained recommendation models, inference outputs, and reproducible metadata for monitoring and lineage.

- Business problem: recommend relevant products to users based on interaction history and item metadata.
- Expected outputs: clean EDA-ready datasets, feature tables, trained model artifact, inference interface.
- Evaluation metrics: Precision@K, Recall@K, HitRate@K, and NDCG@K.
- Implemented models: Retailrocket event-weighted popularity baseline and category/content-based recommender.

3. Data Sources

3.1 DummyJSON API Source

DummyJSON provides API-based products, users, and carts data for a small pipeline demo.

file_path	file_name	top_level_keys	list_key	record_count
data/raw/source=dummyjson/type=carts/ingest_date=2026-04-20/carts_20260420_22...	carts_20260420_224802.json	carts, total, skip, limit	carts	100
data/raw/source=dummyjson/type=carts/ingest_date=2026-04-21/carts_20260421_03...	carts_20260421_031642.json	carts, total, skip, limit	carts	100
data/raw/source=dummyjson/type=carts/ingest_date=2026-04-22/carts_20260422_00...	carts_20260422_002638.json	carts, total, skip, limit	carts	100
data/raw/source=dummyjson/type=carts/ingest_date=2026-04-22/carts_20260422_02...	carts_20260422_020241.json	carts, total, skip, limit	carts	100
data/raw/source=dummyjson/type=products/ingest_date=2026-04-20/products_20260...	products_20260420_224800.json	products, total, skip, limit	products	100
data/raw/source=dummyjson/type=products/ingest_date=2026-04-21/products_20260...	products_20260421_031640.json	products, total, skip, limit	products	100
data/raw/source=dummyjson/type=products/ingest_date=2026-04-22/products_20260...	products_20260422_002636.json	products, total, skip, limit	products	100
data/raw/source=dummyjson/type=products/ingest_date=2026-04-22/products_20260...	products_20260422_020238.json	products, total, skip, limit	products	100

3.2 Retailrocket External Dataset

Retailrocket is the main large-scale dataset. It includes visitor-item events, item properties, and category hierarchy data.

dataset_name	file_path	file_size_mb	row_count	distinct_visitors	distinct_items	distinct_properties	distinct_categories
events	data/external/retailrocket/events.csv	89.872	2756101.0	1407580.0	235061.0		
item_properties_combined	data/external/retailrocket/item_properties_part1.csv; data/external/retailrocket/...	851.865	20275902.0		417053.0	1104.0	
category_tree	data/external/retailrocket/category_tree.csv	0.014	1669.0				1669.0

event	row_count	distinct_visitors	distinct_items	rows_with_transaction_id
view	2664312	1404179	234838	0.0
addtocart	69332	37722	23903	0.0
transaction	22457	11719	12025	22457.0

4. Ingestion, Raw Storage, and Logging

The ingestion layer includes API ingestion for DummyJSON and external batch ingestion for Retailrocket. Data is stored in a structured local data lake. DummyJSON raw files are partitioned by source, type, and ingest date. Retailrocket source data is tracked by DVC as an external dataset.

- DummyJSON ingestion script: `src/ingestion/fetch_dummyjson.py`
- Retailrocket inspection script: `src/ingestion/inspect_retailrocket_external.py`
- Ingestion log evidence: `logs/ingestion_dummyjson.log`
- Raw storage layout: `data/raw/source=dummyjson/type=*/ingest_date=*`
- External storage layout: `data/external/retailrocket`

5. Data Validation and Quality Reports

5.1 DummyJSON Validation

status	check_count
PASS	46
WARN	1

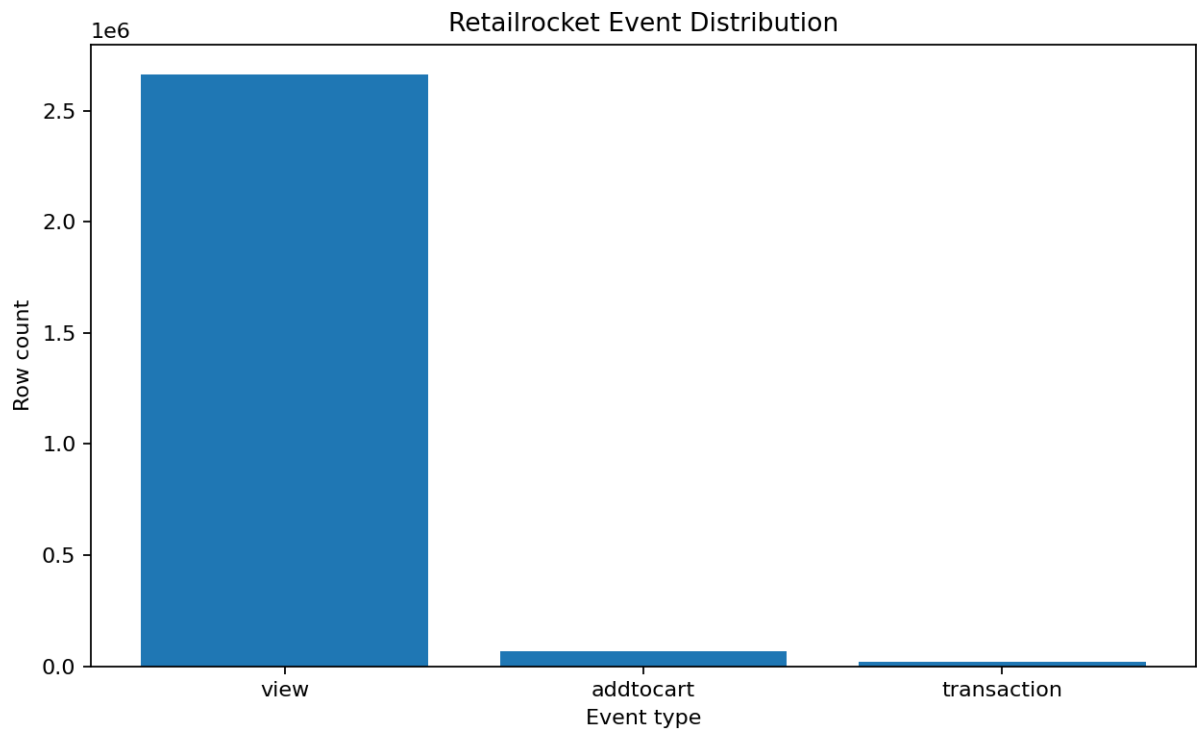
5.2 Retailrocket Validation

status	check_count
PASS	36

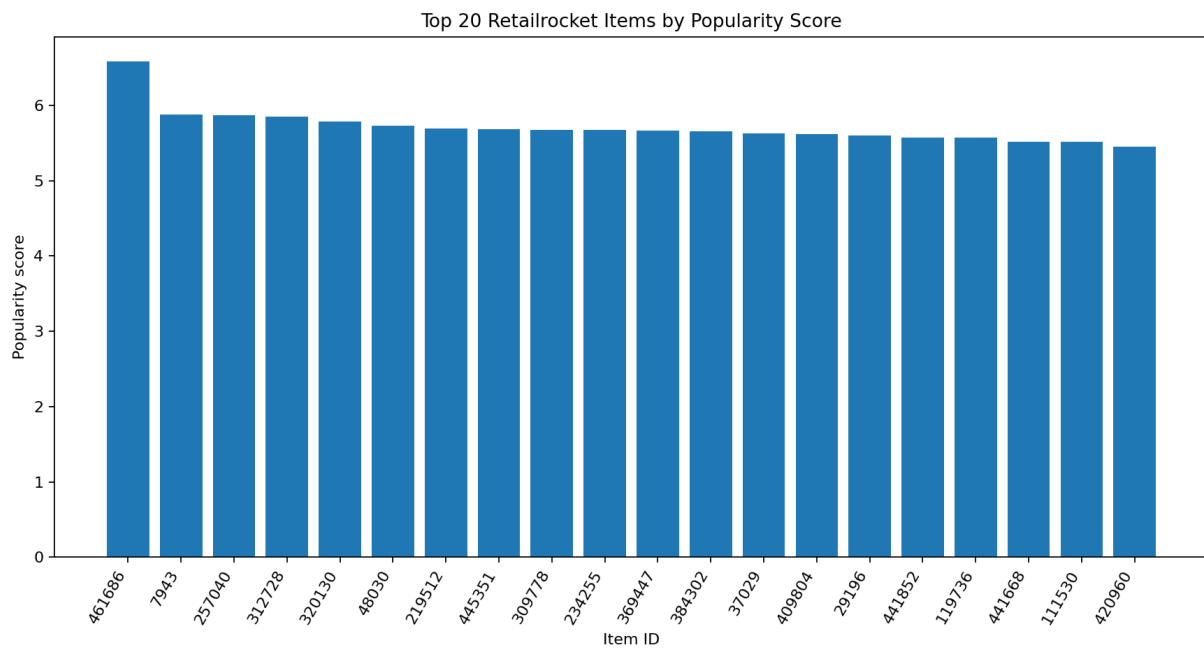
Validation checks include missing values, duplicate IDs, schema mismatch, valid event types, transaction ID consistency, latest item metadata uniqueness, and metadata coverage.

6. Data Preparation and EDA Plots

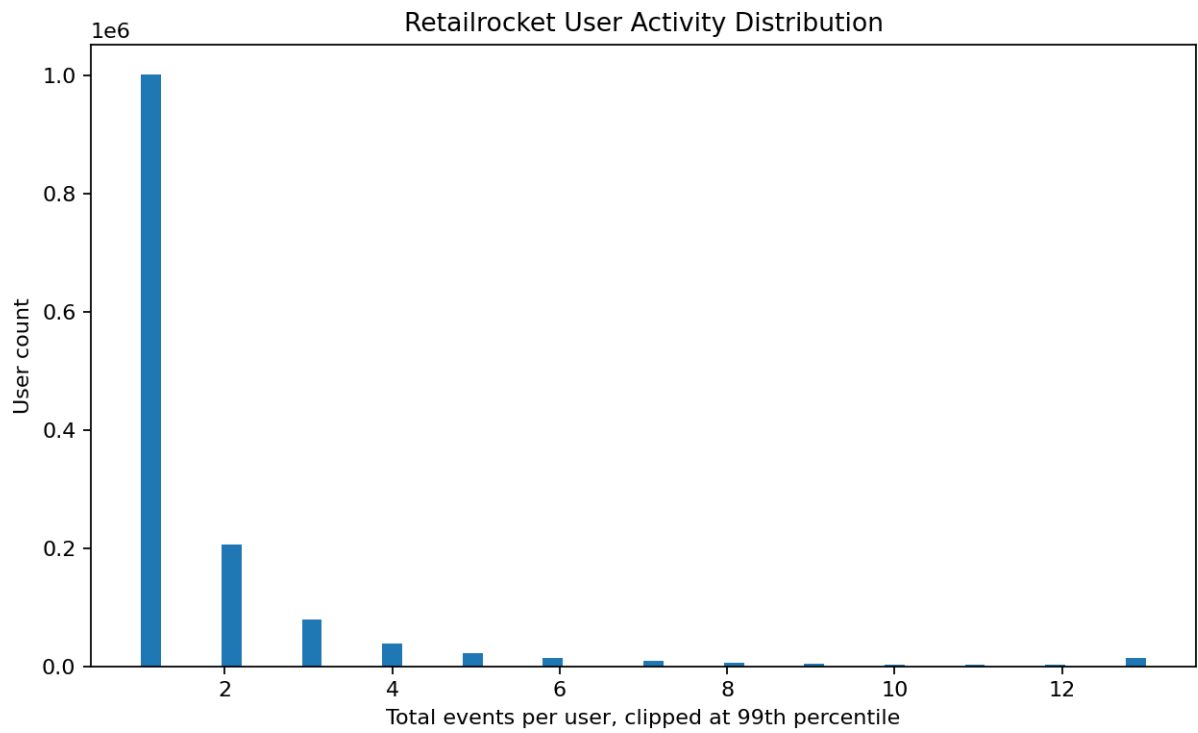
Raw CSV/API data is converted into clean Parquet datasets. EDA artifacts summarize event distribution, item popularity, user activity distribution, and model metrics.



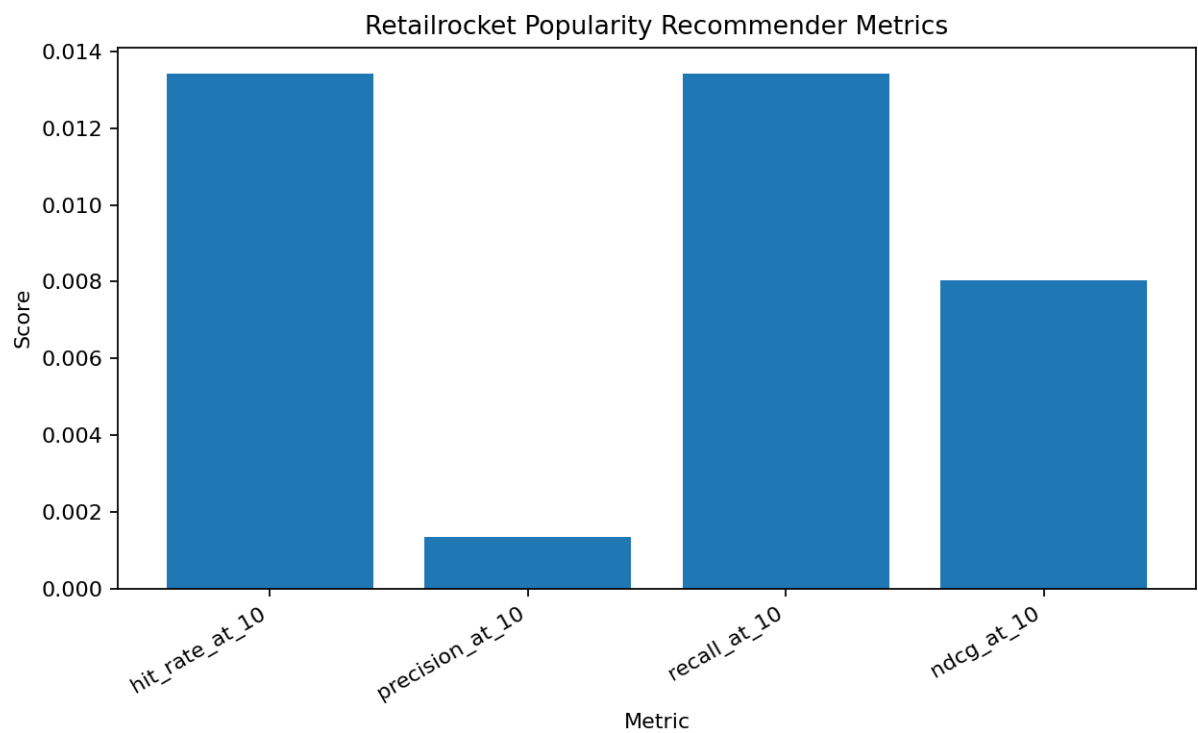
Retailrocket event distribution.



Top Retailrocket items by popularity score.



Retailrocket user activity distribution.



Retailrocket model metrics.

7. SQL Warehouse Schema and Transformation

DuckDB is used as a local analytical warehouse. Staged Parquet files are loaded into staged tables and transformed into mart and feature tables.

schema_name	relation_name	relation_type	row_count	column_count	status
staged	retailrocket_events	BASE TABLE	2756101	8	PASS
staged	retailrocket_category_tree	BASE TABLE	1669	5	PASS

schema_name	relation_name	relation_type	row_count	column_count	status
staged	retailrocket_item_properties_selected	BASE TABLE	2291853	7	PASS
staged	retailrocket_item_category_latest	BASE TABLE	417053	6	PASS
staged	retailrocket_item_availability_latest	BASE TABLE	417053	7	PASS
mart	retailrocket_interactions	VIEW	2756101	9	PASS
mart	retailrocket_user_item_interactions	BASE TABLE	2145179	14	PASS
mart	retailrocket_user_features	BASE TABLE	1407580	12	PASS
mart	retailrocket_item_features	BASE TABLE	235061	18	PASS

7.1 SQL Schema Sample

schema_name	table_name	column_name	ordinal_position	data_type	is_nullable
features	dummyjson_interaction_features	user_id	1	BIGINT	YES
features	dummyjson_interaction_features	item_id	2	BIGINT	YES
features	dummyjson_interaction_features	cart_id	3	BIGINT	YES
features	dummyjson_interaction_features	event_type	4	VARCHAR	YES
features	dummyjson_interaction_features	interaction_weight	5	DOUBLE	YES
features	dummyjson_interaction_features	interaction_price	6	DOUBLE	YES
features	dummyjson_interaction_features	line_total	7	DOUBLE	YES
features	dummyjson_interaction_features	line_discounted_total	8	DOUBLE	YES
features	dummyjson_interaction_features	item_category	9	VARCHAR	YES
features	dummyjson_interaction_features	item_rating	10	DOUBLE	YES
features	dummyjson_interaction_features	item_stock	11	BIGINT	YES
features	dummyjson_interaction_features	item_brand	12	VARCHAR	YES
features	dummyjson_interaction_features	has_catalog_metadata	13	INTEGER	YES
features	dummyjson_interaction_features	_source_system	14	VARCHAR	YES
features	dummyjson_interaction_features	_source_file	15	VARCHAR	YES
features	dummyjson_interaction_features	_prepared_at	16	VARCHAR	YES
features	dummyjson_item_features	item_id	1	BIGINT	YES
features	dummyjson_item_features	title	2	VARCHAR	YES

8. Feature Engineering Logic

Feature engineering creates user-level, item-level, and user-item interaction features for recommendation.

feature_group	feature_or_group	source	logic	used_for
retailrocket_user_features	total_events	mart.retailrocket_interactions	Count all events generated by each user.	User activity representation, training, inference analysis
retailrocket_user_features	unique_items	mart.retailrocket_interactions	Count distinct items interacted with by each user.	User diversity and activity profiling
retailrocket_user_features	view_events, addtocart_events, transaction_events	mart.retailrocket_interactions	Conditional event counts by user and event type.	Intent-strength measurement
retailrocket_user_features	addtocart_event_rate, transaction_event_rate	features.retailrocket_user_features	Event-type count divided by total user event count.	User conversion tendency
retailrocket_item_features	total_events, unique_users	mart.retailrocket_interactions	Count total item interactions and distinct users per item.	Item popularity and candidate ranking
retailrocket_item_features	category_id, parent_category_id	staged.retailrocket_item_category_latest, staged.retailrocket_category_tree	Attach latest item category and parent category metadata.	Content-aware filtering and catalog analysis
retailrocket_item_features	is_available	staged.retailrocket_item_availability_latest	Attach latest availability flag for each item.	Candidate filtering and item metadata enrichment
retailrocket_item_features	addtocart_rate, transaction_rate, cart_to_transaction_rate	features.retailrocket_item_features	Conversion ratios derived from item-level event counts.	Ranking and item quality signals
retailrocket_user_item_features	view_count, addtocart_count, transaction_count	mart.retailrocket_user_item_interactions	Aggregate user-item event counts by event type.	Implicit feedback model training
retailrocket_user_item_features	implicit_label	features.retailrocket_user_item_features	Encode interaction strength: view < addtocart < transaction.	Recommendation target signal
retailrocket_user_item_features	strongest_event_type	features.retailrocket_user_item_features	Select strongest observed user-item event type.	Interpretability and behavioral analysis
dummyjson_feature_registry	user, item, interaction feature views	configs/feature_store/dummyjson_feature_registry.json	Custom registry maps feature views to entities, source paths, and intended us...	Feature store demonstration

9. Feature Store and Feature Metadata

A custom feature registry is implemented to demonstrate feature store concepts. Feature metadata documents feature views, entities, source paths, column roles, data types, and version.

9.1 Sample Feature Retrieval Demonstration

retrieval_type	entity_filter	rows_returned	columns_returned	source_feature_view
user_features	user_id=1	1	14	dummyjson_user_features
item_features	item_id=86	1	21	dummyjson_item_features
interaction_features	user_id=2,item_id=86	1	16	dummyjson_interaction_features

9.2 Feature Metadata Documentation Sample

feature_view_name	entity	source_path	used_for	column_name	data_type	column_role
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	user_id	int64	entity_key
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	total_events	int64	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	unique_items	int64	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	view_events	double	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	addtocart_events	double	feature

feature_view_name	entity	source_path	used_for	column_name	data_type	column_role
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	transaction_events	double	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	total_interaction_score	decimal128(38, 1)	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	avg_interaction_weight	double	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	first_event_timestamp_ms	int64	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	last_event_timestamp_ms	int64	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	first_event_datetime	timestamp[us, tz=UTC]	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	last_event_datetime	timestamp[us, tz=UTC]	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	active_days	int64	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	addtocart_event_rate	double	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	transaction_event_rate	double	feature
retailrocket_user_features	user_id	data/features/source=retailrocket/user_features.parquet	training, batch inference, user profiling	events_per_unique_item	double	feature
retailrocket_item_features	item_id	data/features/source=retailrocket/item_features.parquet	training, candidate ranking, item profiling	item_id	int64	entity_key
retailrocket_item_features	item_id	data/features/source=retailrocket/item_features.parquet	training, candidate ranking, item profiling	total_events	int64	feature
retailrocket_item_features	item_id	data/features/source=retailrocket/item_features.parquet	training, candidate ranking, item profiling	unique_users	int64	feature
retailrocket_item_features	item_id	data/features/source=retailrocket/item_features.parquet	training, candidate ranking, item profiling	view_events	double	feature

10. Data Versioning and Lineage with DVC

DVC is used to track large datasets and model artifacts while Git stores source code, reports, and DVC metadata files. This provides reproducibility without committing heavy files directly to Git.

pipeline_layer	description	dvc_file	tracked_path	md5	size_bytes	nfiles	versioning_tool
external_data	Retailrocket original external CSV dataset	data/external/retailrocket.dvc	retailrocket	19cdee87944c59fc15c51396fe37cd88.dir	987498023	4	DVC
staged_data	DummyJSON staged Parquet data	data/staged/source=dummyjson.dvc	source=dummyjson	4c949baf4d1f10f63deb33acfaee42db.dir	152717	4	DVC
feature_data	DummyJSON feature Parquet data	data/features/source=dummyjson.dvc	source=dummyjson	c49242017c40e790059bbf13fa924e98.dir	41299	3	DVC
model_artifact	DummyJSON trained popularity model	models/dummyjson.dvc	dummyjson	26596e9e6569b24edc95075f03ae88af.dir	29199	1	DVC

pipeline_layer	description	dvc_file	tracked_path	md5	size_bytes	nfiles	versioning_tool
staged_data	Retailrocket staged Parquet data	data/staged/source=retailrocket.dvc	source=retailrocket	2896e046e1755c12b6e21c445da0a2af.dir	80572198	5	DVC
feature_data	Retailrocket feature Parquet data	data/features/source=retailrocket.dvc	source=retailrocket	8b21fd5e8ab42536cd855a1f96b85c37.dir	147337179	3	DVC
model_artifact	Retailrocket trained popularity model	models/retailrocket.dvc	retailrocket	ddfd4f90db7b60b49908097e66e97917.dir	1227304	2	DVC

10.1 Repository Structure and Dataset Versioning

path	purpose	versioning	example_files
configs/feature_store/	Custom feature registry configuration	Git	dummyjson_feature_registry.json
data/external/	External source datasets	DVC metadata in Git	retailrocket.dvc
data/raw/	Raw API ingested data	Ignored by Git; reproducible through ingestion	source=dummyjson/type=*/ingest_date=*
data/staged/	Cleaned staged Parquet datasets	DVC metadata in Git	source=dummyjson.dvc, source=retailrocket.dvc
data/features/	ML feature datasets	DVC metadata in Git	source=dummyjson.dvc, source=retailrocket.dvc
data/warehouse/	Local DuckDB analytical warehouse	Generated artifact, ignored by Git	recomart.duckdb
models/	Trained model artifacts	DVC metadata in Git	dummyjson.dvc, retailrocket.dvc
reports/	Reports, plots, screenshots, summaries	Git for lightweight files	final_project_report.md, plots/, screenshots/
sql/	Warehouse and feature transformation SQL	Git	retailrocket_warehouse.sql, retailrocket_features.sql
src/	Pipeline source code organized by stage	Git	ingestion/, validation/, preparation/, transformation/, training/, serving/
orchestration/	Prefect pipeline definitions	Git	dummyjson_pipeline.py, retailrocket_pipeline.py

11. Model Training and Evaluation

The Retailrocket modeling stage includes two recommenders: an event-weighted global popularity baseline and a category/content-based filtering model. Event weights are: view = 1, addtocart = 3, transaction = 5. A time-based train/test split is used.

model_name	model_type	top_k	test_window_days	evaluation_sample_users	min_time_stamp_ms	max_time_stamp_ms	cutoff_time_stamp_ms	min_event_datetime	max_event_datetime	train_event_count	test_event_count
retailrocket_popularity_recommender	event_weighted_global_popularity	10	14	10000	1430622004384	1442545187788	1441335587788	2015-05-03 08:30:04.384000+05:30	2015-09-18 08:29:47.788000+05:30	2509138	246963

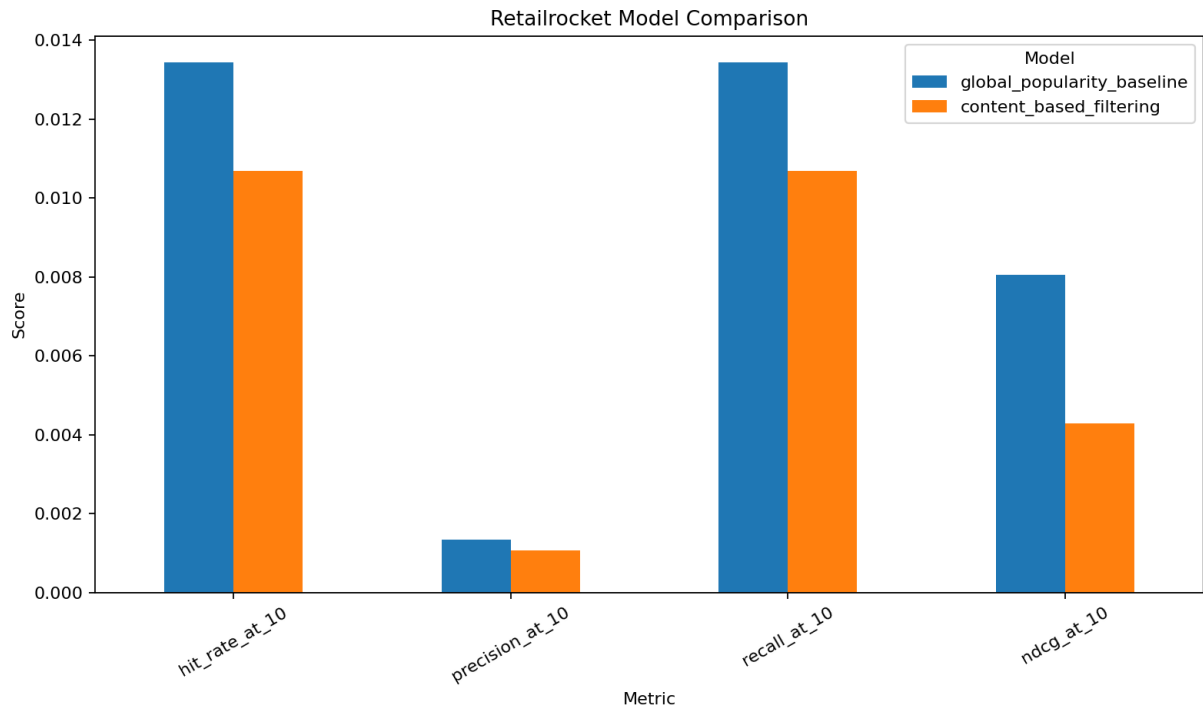
11.2 Content-Based Recommender

A second Retailrocket model was implemented using category/content-based filtering. It builds user profiles from historical item categories and parent categories, then scores candidate items using category similarity, availability metadata, conversion signals, and a popularity prior. This directly addresses the assignment requirement for content-based filtering.

model_name	model_type	top_k	test_window_days	evaluation_sample_users	min_time_stamp_ms	max_time_stamp_ms	cutoff_time_stamp_ms	candidate_item_count	test_target_users	user_profile_count	evaluated_users
retailrocket_content_based_recommender	category_content_based_filtering	10	14	10000	1430622004384	1442545187788	1441335587788	5000	3648	517	3648

11.3 Model Comparison

model_family	model_name	model_type	top_k	evaluated_users	hits	hit_rate_at_10	precision_at_10	recall_at_10	ndcg_at_10
global_popularity_baseline	retailrocket_popularity_recommender	event_weighted_global_popularity	10	3648	49	0.0134320175438596	0.0013432017543859	0.0134320175438596	0.0080460627572333
content_based_filtering	retailrocket_content_based_recommender	category_content_based_filtering	10	3648	39	0.0106907894736842	0.0010690789473684	0.0106907894736842	0.0042938331780684



Comparison of popularity baseline and content-based recommender metrics.

Retailrocket performance: HitRate@10 = 0.013432, Precision@10 = 0.001343, Recall@10 = 0.013432, NDCG@10 = 0.008046.

NDCG@10 is ranking-aware: a relevant item at rank 1 receives more credit than one at rank 10.

12. MLflow Tracking Evidence

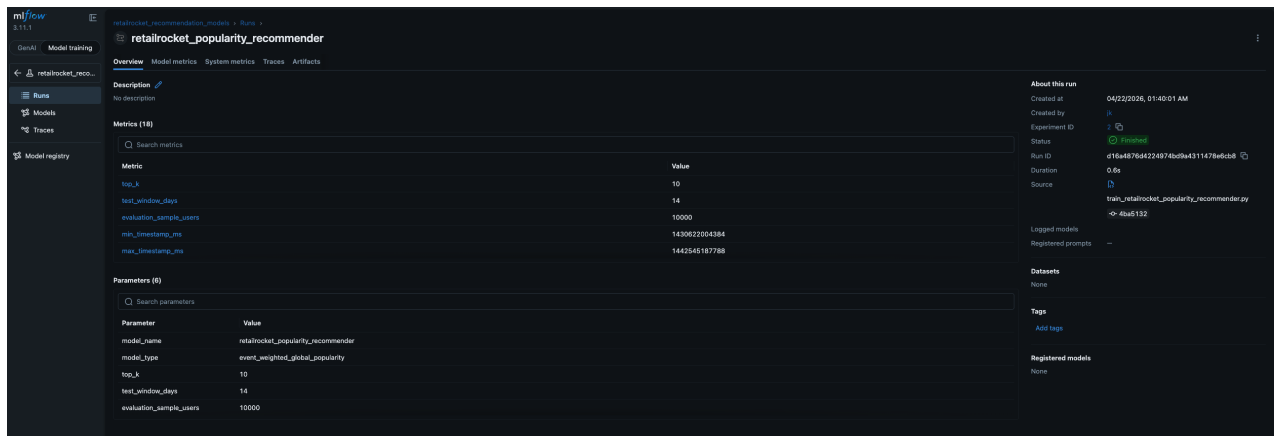
MLflow stores run IDs, parameters, metrics, and artifacts. Screenshots below show successful experiment tracking for DummyJSON and Retailrocket.

Name	Time created	Last modified	Description	Tags
retailrocket_recommendation_models	04/21/2026, 11:37:09 PM	04/21/2026, 11:37:09 PM	-	-
dummyjson_recommendation_baselines	04/21/2026, 02:52:49 AM	04/21/2026, 02:52:49 AM	-	-
Default	04/21/2026, 02:52:49 AM	04/21/2026, 02:52:49 AM	-	-

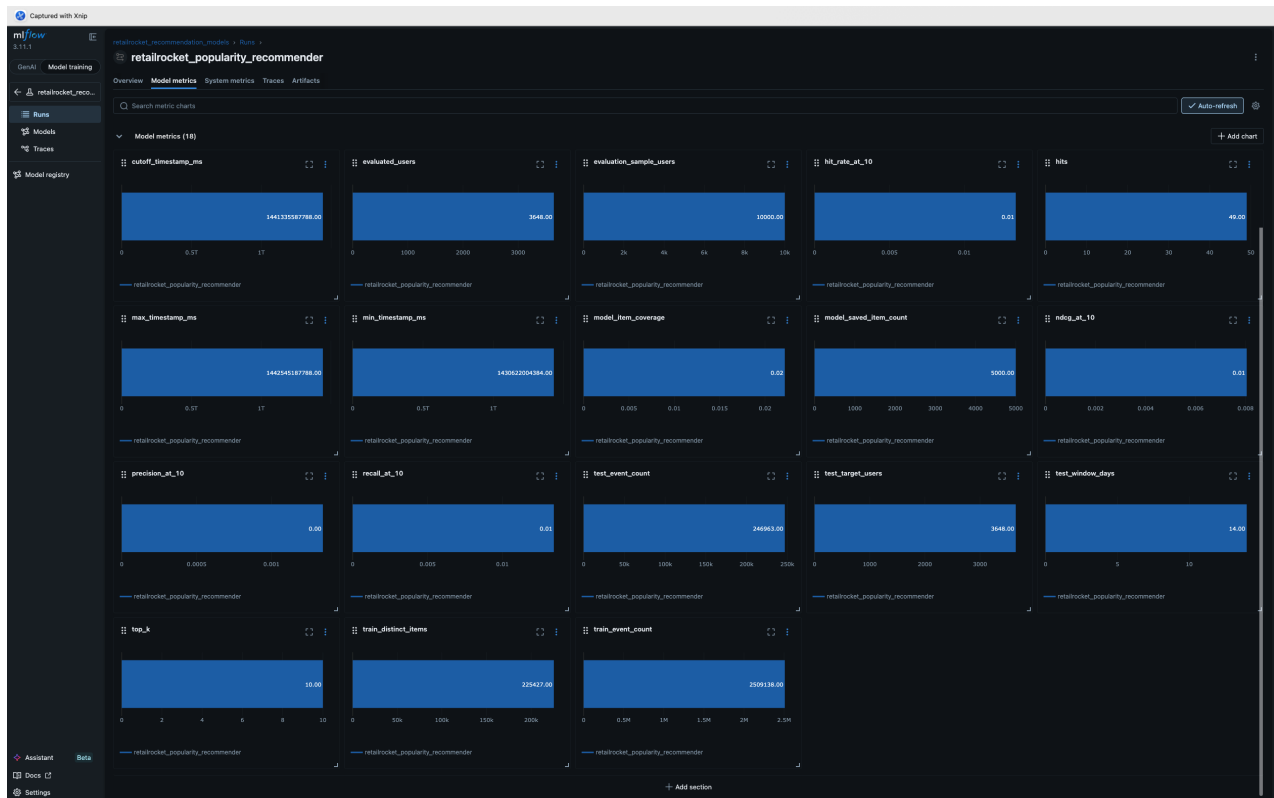
MLflow experiments list.

Run Name	Created	Dataset	Duration	Source	Models	hit_rate_at_10	ndcg_at_10	precision_at_10	recall_at_10	test_event_count	test_target_users	train_event_count
retailrocket_popularity_recommender	3 minutes ago	-	0.6s	train_retailrocket_popularity_recommender.py	-	0.0134320175438596	0.008046062757233324	0.00134320175438596	0.0134320175438596	246963	3648	2509138
retailrocket_popularity_recommender	1 hour ago	-	67ms	train_retailrocket_popularity_recommender.py	-	0.0134320175438596	-	0.00134320175438596	0.0134320175438596	246963	3648	2509138
retailrocket_popularity_recommender	1 hour ago	-	71ms	train_retailrocket_popularity_recommender.py	-	0.0134320175438596	-	0.00134320175438596	0.0134320175438596	246963	3648	2509138
retailrocket_popularity_recommender	1 hour ago	-	79ms	train_retailrocket_popularity_recommender.py	-	0.0134320175438596	-	0.00134320175438596	0.0134320175438596	246963	3648	2509138
retailrocket_popularity_recommender	2 hours ago	-	65ms	train_retailrocket_popularity_recommender.py	-	0.0134320175438596	-	0.00134320175438596	0.0134320175438596	246963	3648	2509138

Retailrocket runs table with metrics.



Retailrocket run overview.



Retailrocket metric details.

Run Name	Created	Dataset	Duration	Source	Models
popularity_recommender	1 hour ago	-	68ms	train_popularity_recommender.py	-
popularity_recommender	22 hours ago	-	68ms	train_popularity_recommender.py	-
popularity_recommender	22 hours ago	-	68ms	train_popularity_recommender.py	-
popularity_recommender	22 hours ago	-	47ms	train_popularity_recommender.py	-

DummyJSON runs table.

Run Name	Created	Dataset	Duration	Source	Metrics
retailrocket_content_based_recommender	4 minutes ago	-	80ms	train_retailrocket_content_recommender.py	-
retailrocket_popularity_recommender	59 minutes ago	-	68ms	train_retailrocket_popularity_recommender.py	-
retailrocket_popularity_recommender	1 hour ago	-	78ms	train_retailrocket_popularity_recommender.py	-
retailrocket_popularity_recommender	1 hour ago	-	0.6s	train_retailrocket_popularity_recommender.py	-
retailrocket_popularity_recommender	2 hours ago	-	67ms	train_retailrocket_popularity_recommender.py	-
retailrocket_popularity_recommender	2 hours ago	-	71ms	train_retailrocket_popularity_recommender.py	-
retailrocket_popularity_recommender	3 hours ago	-	78ms	train_retailrocket_popularity_recommender.py	-
retailrocket_popularity_recommender	3 hours ago	-	65ms	train_retailrocket_popularity_recommender.py	-

Retailrocket content-based recommender run in MLflow.

13. Inference Demo

The inference script loads the model and generates recommendations for default active users, one user, or custom comma-separated users. Previously seen items are filtered.

model_name	model_type	requested_user_count	top_k	recommendation_rows	distinct_recommended_items	model_saved_item_count	training_hit_rate_at_10	training_precision_at_10	training_recall_at_10
retailrocket_popularity_recommender	event_weighted_global_popularity	3	5	15	5	5000	0.0134320175438596	0.0013432017543859	0.0134320175438596

14. Pipeline Orchestration

Prefect orchestrates the pipeline. The Retailrocket flow runs inspection, preparation, validation, DuckDB loading, feature engineering, model training, and inference.

14.1 DummyJSON Orchestration

step_order	step_name	status	duration_seconds
1	ingest_dummyjson_api	PASS	3.517
2	inspect_dummyjson_raw	PASS	0.459
3	validate_dummyjson_raw	PASS	0.374
4	prepare_dummyjson_staged	PASS	0.457
5	validate_dummyjson_staged	PASS	0.437
6	load_dummyjson_to_duckdb	PASS	0.729
7	build_dummyjson_features	PASS	0.46
8	retrieve_dummyjson_features	PASS	0.425
9	train_popularity_recommender	PASS	2.759
10	run_popularity_inference	PASS	0.399

14.2 Retailrocket Orchestration

step_order	step_name	status	duration_seconds
1	inspect_retailrocket_external	PASS	4.668
2	prepare_retailrocket_staged	PASS	3.435
3	validate_retailrocket_staged	PASS	0.643
4	load_retailrocket_to_duckdb	PASS	5.156
5	build_retailrocket_features	PASS	4.57
6	train_retailrocket_popularity_recommender	PASS	24.038
7	run_retailrocket_inference	PASS	0.585

15. Reproducibility Workflow

- Activate environment: `conda activate recomart`
- Check version state: `git status` and `dvc status`
- Restore tracked artifacts if remote is configured: `dvc pull`
- Run DummyJSON flow: `python -m orchestration.dummyjson_pipeline`
- Run Retailrocket flow: `python -m orchestration.retailrocket_pipeline`
- Open MLflow UI using the tracking URI from Python: `mlflow ui --backend-store-uri "$TRACKING_URI" --port 5001`

16. Limitations and Future Work

- Current Retailrocket models are lightweight baselines; future work can add deeper personalized collaborative filtering.

- Future work can add item-item collaborative filtering or matrix factorization.
- Candidate generation can be improved using category and availability filtering.
- A FastAPI serving endpoint and Streamlit monitoring dashboard can be added.
- A scheduled Prefect deployment can automate periodic refreshes.

17. Conclusion

The project implements a complete and reproducible ML data management pipeline for recommendation systems using Python, DuckDB, Parquet, DVC, MLflow, and Prefect. It satisfies the required stages from ingestion through orchestration and produces source code, reports, screenshots, and a final PDF.